

INTRODUCTION

Imagine your production application going down on a Friday afternoon. Within seconds, your monitoring dashboards flash red, alerts hit your phone, and your team pinpoints the exact service failure using detailed logs and traces. This is just a glimpse of what observability can do.

Yet, many businesses and developers treat production systems like high-tech labs but often put no attention to internal toolchains.

In a recent discussion on Software Engineering Radio, Brian Demers and Giovanni Asproni explored a critical shift: the realization that the toolchain is itself a production system. If the assembly line stops, the business stops.

NOT EVERY FAILURE IS A CODE FAILURE

One of the most useful ideas from the discussion is that observability is not just about collecting more data. Its real value is helping teams understand what kind of problem they are facing. When a build fails, the cause may be in the code itself, in the toolchain, or in the infrastructure around it. Being able to tell those cases apart saves time and leads to better decisions, because teams stop treating every pipeline issue as if it came from bad code.

EPHEMERAL CI AND WHY IT MATTERS

A good example of this is ephemeral CI. In systems such as GitHub-hosted CI, each job starts in a fresh environment instead of reusing the same machine over time. That improves consistency, but it also means local state is not automatically preserved. As a result, dependencies may need to be downloaded again unless cache is restored, which can increase build time, network usage, and cost. In those cases, observability helps show that the problem is not necessarily the code, but the environment in which the pipeline is running.

THE AI TSUNAMI AND THE END OF THE "HUMAN EYEBALL"

There is an immediate pressure for better observability coming from the explosion of AI-assisted development. With tools like GitHub Copilot, the volume of code entering repositories is making it impossible for human code reviewers to properly go through all of it.

As Demers points out, if a project receives 10 changes a week, a lead developer can review them thoroughly. If AI helps that team 10X that, the "human eyeball" factor becomes a bottleneck. We are moving toward a world where we need automated "quality gates." Observability allows us to monitor these gates, ensuring that even if

code is being generated at 10x speed, its provenance and its impact on build stability are tracked in real-time.

THE BUSINESS CASE: STOP THROWING DOLLARS OUT THE WINDOW

For many managers, "observability" sounds like an added expense. However, the data proves it is a cost-saver. The podcast highlights three major areas of ROI:

1. **Risk Mitigation:** Much like production monitoring, toolchain observability is about capacity planning. It allows you to see failure coming before it halts the entire engineering department and thus reduces the time it takes to solve the failure.
2. **Productivity increase:** Given that observability decreases the resolution cycle's time, it leads to an overall increase in productivity as people spend less time fixing errors. This all contributes to a shorter time-to-market which in term makes the team more competitive.
3. **Infrastructure Optimization:** Many companies over-provision their infrastructure. Without data, they pay for 100% of a virtual machine's CPU while the build only utilizes 20%. Observability pinpoints this waste, allowing teams to save huge amounts of money by knowing exactly where to allocate resources.

THE COST OF A SEMICOLON: CONTEXT SWITCHING

One of the most profound concepts discussed is the impact of "feedback loops" on the human brain. Science suggests that it takes a developer roughly 10 minutes to "context switch" away from a task and another 10 minutes to switch back.

If a build takes 20 minutes to fail because of a missing semicolon, the developer has already checked their email, joined a Slack thread, or grabbed a coffee. They have lost their "flow." By the time they see the failure, they have to spend another 10 minutes remembering what they were even trying to do. Observability helps teams identify these long feedback loops, allowing engineers to trim the fat and keep developers in the "productive zone."

CULTURAL SHIFT: FROM "NOT MY PROBLEM" TO EMPOWERMENT

In many large organizations, there is a silo between the "CI/CD Team" and the "Developers." When a build fails, developers often just open a ticket for the "CI/CD Team" to fix it.

Observability democratizes data. When a developer has a detailed view of why their specific build failed, they can "self-select" and fix the issue themselves reducing the ticket load on infrastructure teams.

WHERE TO START: THE INCREMENTAL PATH

You don't need to monitor everything on day one. The experts suggest a simple starting point: Build Time.

Start by measuring how long it takes for a build to fail. If it fails in 30 seconds, that's a success of the system (early feedback). If it consistently fails after 45 minutes, that is a system failure. Once you have a baseline for time, you can move into more complex areas like "Flaky Test" tracking and "Code Provenance."

CONCLUSION

Our toolchains are the production infrastructure that powers software delivery. By applying the same observability rigor to our builds that we apply to our websites, we can reduce cloud waste, mitigate AI-related risks, and—most importantly—make developers' lives better.

As the industry moves toward more complex, AI-driven, and decentralized repositories, the companies that manage their toolchains with data will be the ones that out-pace the competition.